

BONUS - Part 3, but Git lives *INSIDE* the cluster

1/4

one k3d cluster 'iot-bonus' - 3 namespaces - Argo CD pulls from a GitLab running next to it

YOUR MACHINE

Browser
http://gitlab.local/

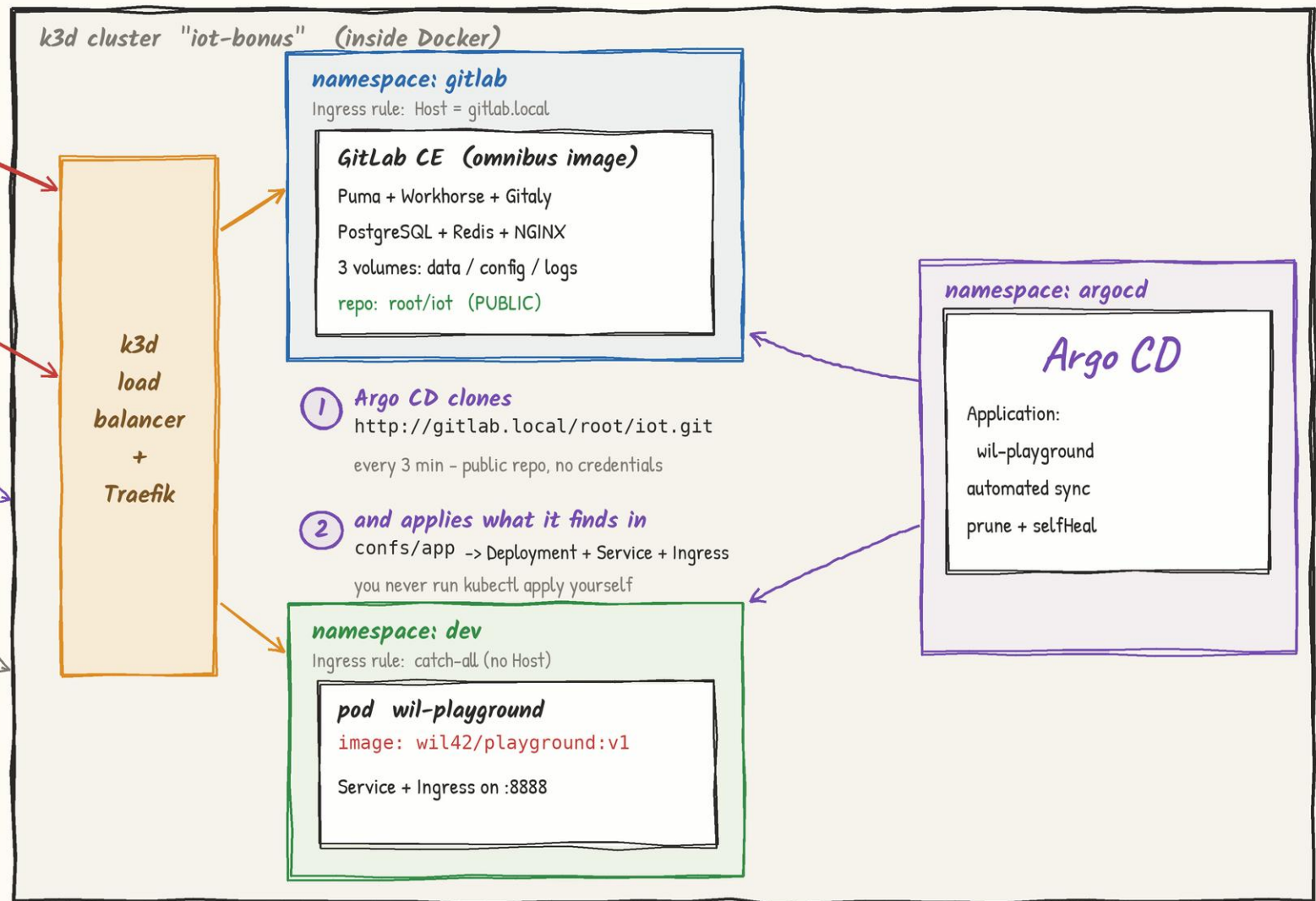
:80

Browser / curl
http://localhost:8888/

:8888

Browser (Argo CD UI)
https://localhost:8080
via kubectl port-forward 8080:443

Your shell
kubectl / k3d / git
straight to the kube-api (kubeconfig)



One URL, two resolvers

host: /etc/hosts -> 127.0.0.1:80 -> Traefik
cluster: CoreDNS rewrite -> gitlab.gitlab.svc
so browser, git push and Argo CD all use the same URL.

Why omnibus and not the Helm chart

since chart v10.0.0 it needs an EXTERNAL Postgres, Redis and S3.
the omnibus image ships them all, always the latest GitLab.

Before you start

~4 GB free RAM, ~5 GB disk.
ports 80 + 8888 free:
k3d cluster delete iot-cluster

HOW TO RUN IT

2/4

one script does everything - the only hard part is waiting for GitLab

1 Free the ports

GitLab wants :80, the app wants :8888 (Part 3 holds it)

```
bash
$ k3d cluster delete iot-cluster
```

2 Go into the bonus folder

every path below is relative to it

```
bash
$ cd bonus
```

3 Run the one and only script

root is needed: it writes /etc/hosts and binds port 80

```
bash
$ sudo ./scripts/setup.sh
```

4 Watch GitLab boot (the long part)

open a second terminal and follow the pod

```
bash
$ kubectl -n gitlab get pods -w
NAME      READY   STATUS    <- reconfigure + migrations
gitlab-0  0/1     Running
gitlab-0  1/1     Running   <- ready, keep going
```

Be patient (this is normal)

first boot: 15 to 25 min - the image is 1.5 GB, then omnibus runs gitlab-ctl reconfigure and the database migrations. Later boots ~5 min. setup.sh waits for you and prints every URL at the end.

what setup.sh does for you

- 1 installs docker, kubectl, k3d and git if they are missing
- 2 creates the cluster iot-bonus: host :80 and :8888 -> Traefik
- 3 creates the 3 namespaces: argocd, dev, gitlab
- 4 makes gitlab.local resolve: /etc/hosts + CoreDNS rewrite
- 5 deploys GitLab CE (omnibus) and waits until the pod is ready
- 6 creates the PUBLIC project root/iot, pushes confs/app into it
- 7 installs Argo CD + the Application -> auto-sync starts

and it ends by printing this:

```
setup.sh
GitLab UI : http://gitlab.local/
           root / Iot42-Bonus-Pass
Argo CD UI : https://localhost:8080
           admin / <password printed by the script>
App       : http://localhost:8888/
```

lost the Argo CD password?

```
kubectl -n argocd get secret argocd-initial-admin-secret \
-o jsonpath="{.data.password}" | base64 -d
```

the app answers on <http://localhost:8888/> as soon as Argo CD syncs

HOW TO TEST IT

3/4

five checks you can show, then the demo that actually proves it

✓ the three namespaces exist, with their pods

```
bash
$ kubectl get ns
argocd    Active
dev       Active
gitlab    Active

$ kubectl get pods -n gitlab
gitlab-0      1/1 Running

$ kubectl get pods -n dev
wil-playground-.. 1/1 Running
```

✓ GitLab answers on its own URL

```
http://gitlab.local/

login: root    password: lot42-Bonus-Pass

the repository is there: root/iot (public), with confs/app inside
```

✓ Argo CD is Synced / Healthy from the LOCAL GitLab

```
bash
$ kubectl -n argocd get app wil-playground
NAME          SYNC STATUS  HEALTH STATUS
wil-playground Synced      Healthy

$ kubectl -n argocd get app wil-playground \
  -o jsonpath='{.spec.source.repoURL}'
http://gitlab.local/root/iot.git
```

✓ the application answers, in v1

```
bash
$ curl http://localhost:8888/
{"status":"ok", "message": "v1"}
```

✓ or run every check above in one shot:

```
$ ./testKiller.sh          # all the checks
$ ./testKiller.sh --cd     # + the full v1 <-> v2 test
```

THE REAL TEST: you change Git, the cluster follows by itself



```
the short way

$ sudo ./scripts/switch_version.sh v2
... pushing v2 to GitLab, refreshing Argo CD
Application is now running v2.
$ sudo ./scripts/switch_version.sh v1 # rolls back
```

The point

you never run kubectl apply: the only source of truth is the repo hosted by the GitLab pod next door.

CLEANUP, and what to do when it breaks

4/4

plus the three questions you will be asked about this bonus

CLEANUP

```
bash
$ sudo ./scripts/cleanup.sh
```

- deletes the k3d cluster iot-bonus (GitLab data included)
- removes the gitlab.local line from /etc/hosts
- removes bonus/.gitlab-token

WHEN IT BREAKS

"port 80 / 8888 is already in use"

another cluster holds them - k3d cluster delete iot-cluster, then re-run setup.sh

GitLab shows 502, or the pod stays 0/1

it is still reconfiguring: `kubectl -n gitlab logs -f gitlab-0`. It needs ~4 GB of RAM.

Argo CD: Unknown / "failed to clone"

gitlab.local does not resolve in the cluster: `kubectl -n kube-system rollout restart deploy coredns`

switch_version.sh: "no access token"

run setup.sh first - it writes bonus/.gitlab-token used to push

the 5 minute demo, in order

1. `kubectl get ns + get pods -n gitlab -n dev`
2. GitLab UI: the repo root/iot and its confs/app
3. Argo CD UI: source = `http://gitlab.local/root/iot.git`, Synced
4. `curl :8888 -> v1`
5. `switch_version.sh v2 -> curl -> v2`

QUESTIONS YOU WILL BE ASKED

Q Why not the official Helm chart?

A Since chart v10.0.0 it refuses to render without an EXTERNAL PostgreSQL, Redis and object storage. The gitlab/gitlab-ce omnibus image ships that whole stack and is always the latest GitLab release.

Q Why is the project public?

A So Argo CD clones it anonymously: no credential stored in the cluster, exactly like the GitHub repository of Part 3.

Q How can one URL work on both sides?

A On the host, /etc/hosts. Inside the cluster, a CoreDNS rewrite. The same `http://gitlab.local/root/iot.git` works in the browser, in git push and in Argo CD.

```
what is in the folder
bonus/
  confs/app/      deployment.yaml (v1) service.yaml ingress.yaml
  confs/gitlab/   statefulset.yaml service.yaml ingress.yaml
                  coredns-custom.yaml
  confs/argocd/   application.yaml (source = the local GitLab)
  scripts/       setup.sh seed_gitlab.sh switch_version.sh cleanup.sh
  testKiller.sh  every check of page 3, in one run
```